UAV Swarm Fault Detection & Isolation System

Autonomous Multi-Agent Agentic AI Architecture with Real-Time Mission Control Dashboard

Project	UAV Swarm FDI — Multi-Agent Autonomous Safety System
Domain	Unmanned Aerial Vehicles (UAVs) / Autonomous Swarm Robotics
Core Technology	LangGraph Agentic AI, Control Barrier Functions (CBF), ML-FDI
Version	1.0 (Production-Ready Simulation)
Date	October 23, 2026
Platform	Python 3.x / Tornado / Three.js / ROS 2

Table of Contents

1. Abstract & Project Overview

2. Problem Statement & Motivation

3. System Architecture

3.1 High-Level Architecture Overview

3.2 10-Node LangGraph Agent Pipeline

3.3 SwarmBus — Decentralized Communication

3.4 Control Barrier Functions (CBF-QP)

3.5 Machine Learning FDI Model

4. No-Fly Zone Geofencing & Dynamic Rerouting

4.1 NFZ Configuration

4.2 Multi-Layer Avoidance Strategy

4.3 Dynamic Bypass Waypoint Algorithm

5. Two-Layer Guardrail Safety System

5.1 Layer 1 — Soft Policy Guardrail

5.2 Layer 2 — CBF-QP Hard Safety Filter

6. Swarm Coordination & Intelligence

6.1 Leader Election Protocol

6.2 Task Delegation & Handoff

6.3 Inter-Agent Collision Avoidance

7. Energy & Battery Management

8. Risk Assessment & Agentic Reasoning

9. Mission Control Web Dashboard

9.1 3D Three.js Visualization

9.2 2D Radar Map with Zoom/Pan

9.3 Telemetry Cards & Live Status

9.4 Fault Injection Console

10. Terminology & Glossary

10.1 Flight Actions

10.2 FDI Fault Labels

10.3 Energy Alerts

10.4 Guardrail Statuses

10.5 Swarm Roles

11. Technology Stack

12. File Structure & Codebase

13. How to Run the System
14. Future Enhancements
15. Conclusion

1. Abstract & Project Overview

This project implements a **fully autonomous multi-agent UAV swarm system** capable of real-time **Fault Detection and Isolation (FDI)** during autonomous flight missions. The system deploys a fleet of 4 (or more, via horizontal scaling) UAV agents, each operating as an independent **LangGraph-based agentic AI pipeline** that processes sensor telemetry, detects faults using machine learning, reasons about risk, coordinates with peer agents, and executes safety-critical decisions — all without human intervention.

Each UAV agent runs a **10-node state graph pipeline** every simulation tick (~1 second), providing continuous monitoring, fault detection (6 fault classes), energy management, No-Fly Zone geofencing, and swarm coordination. The system features a **two-layer guardrail safety architecture**: a soft policy guardrail (Layer 1) that enforces operational rules, and a **Control Barrier Function (CBF) quadratic programming safety filter** (Layer 2) that mathematically guarantees collision avoidance and geofence compliance.

A **web-based Mission Control Dashboard** provides real-time visualization through an interactive 3D Three.js scene and a 2D radar map with zoom/pan controls, live telemetry cards for each UAV, fault injection console, and swarm status overview. The entire system is designed to simulate what a real-world autonomous drone fleet management and safety assurance system would look like.

> ■ ***Key Innovation:*** *This project combines Agentic AI (LangGraph state machines), formal safety verification (Control Barrier Functions), and Machine Learning fault detection into a unified, production-ready autonomous swarm architecture.*

2. Problem Statement & Motivation

As autonomous UAV operations scale from single-drone missions to multi-agent swarm deployments, several critical challenges emerge:

- **Sensor Fault Detection:** UAVs rely on GPS, IMU (accelerometer, gyroscope), barometer, and compass sensors. Hardware failures, electromagnetic interference, or environmental factors can corrupt sensor readings. Without real-time detection, a faulty sensor can cause crashes, navigation errors, or mission failure.
- **Autonomous Decision Making:** In remote or communication-degraded environments, UAVs must make safety-critical decisions independently — when to abort, reroute, land, or hand off tasks to peers.
- **No-Fly Zone Compliance:** UAVs must never enter restricted airspace (hospitals, airports, military zones). Violations can result in legal consequences, safety hazards, or drone destruction. The system must proactively detect and avoid NFZ boundaries, not just react after entry.
- **Swarm Coordination:** Multiple UAVs must avoid colliding with each other, dynamically reassign tasks when a peer becomes degraded, and maintain formation without centralized control.
- **Formal Safety Guarantees:** Simple rule-based geofencing is insufficient. The system needs mathematically provable safety guarantees — Control Barrier Functions provide this by ensuring the system state remains within a defined safe set at all times.

This project addresses all these challenges in a unified architecture, demonstrating how **Agentic AI**, **Machine Learning**, and **Formal Safety Methods** can work together to create a robust, autonomous UAV swarm system.

3. System Architecture

3.1 High-Level Architecture Overview

The system consists of three major layers that work together to create a fully autonomous swarm:

Layer	Component	Purpose
Intelligence Layer	LangGraph 10-Node Agent Pipeline	Each UAV runs an independent AI agent that processes telemetry, detects faults, re
Safety Layer	Two-Layer Guardrail System (L1 + L2 CBF)	Layer 1 enforces soft policy rules. Layer 2 uses CBF-QP optimization to mathematic
Communication Layer	SwarmBus + ROS 2 + MAVLink	Decentralized peer-to-peer message broker for heartbeats, task handoff, and leader
Visualization Layer	Mission Control Web Dashboard	Tornado web server serving a single-page app with 3D Three.js scene, 2D radar, liv
ML Detection Layer	ExtraTrees FDI Classifier	Pre-trained 6-class fault classifier (Normal, GPS, Compass, Accelerometer, Barome

Table 1: System Architecture Layers

3.2 The 10-Node LangGraph Agent Pipeline

Every UAV agent executes a **LangGraph StateGraph** pipeline on every simulation tick (approximately once per second). The pipeline is a directed acyclic graph (DAG) of 10 processing nodes, where the output state of each node flows into the next. This design ensures that every decision is made through a structured, auditable, multi-stage reasoning process.

#	Node Name	Function	Description
1	Telemetry	node_collect_telemetry	Reads sensor data (position, velocity, altitude, battery), advances waypoint navigati
2	Energy Monitor	node_monitor_energy	Tracks battery percentage, computes drain rate, classifies energy state (NOMINAL /
3	FDI Inference	node_run_fdi	Runs the ExtraTrees ML model on current sensor features to detect faults. Classifies
4	Risk Assessment	node_compute_risk	Calculates composite risk score (0.0–1.0) based on fault detection, battery level, spe
5	Agentic Reasoning	node_agentic_reasoning	Core AI decision engine. Based on risk, faults, energy, and NFZ state, decides actio
6	Swarm Coordination	node_swarm_coordination	Handles inter-agent communication: processes handoff requests from degraded pee

#	Node Name	Function	Description
7	Guardrail L1	node_guardrail_l1	Soft policy guardrail: validates altitude bounds (2m–120m), speed limits (≤5 m/s), ba
8	Guardrail L2	node_guardrail_l2	Hard safety filter using Control Barrier Functions (CBF) with Quadratic Programming
9	Execute Action	node_execute_action	Applies the finalized (guardrail-filtered) action. In simulation, updates the state objec
10	Finalize	node_finalize	Broadcasts heartbeat to SwarmBus with current state summary (position, velocity, b

Table 2: LangGraph 10-Node Pipeline — Each UAV Agent

■ *The pipeline is compiled via* `build_uav_graph()` *using LangGraph's* `StateGraph` *API. Edges connect START → telemetry → energy → fdi → risk → reasoning → swarm → guardrail_l1 → guardrail_l2 → execute → finalize → END.*

3.3 SwarmBus — Decentralized Communication Protocol

The **SwarmBus** is a thread-safe, decentralized message broker that enables peer-to-peer communication between all UAV agents without a central server. It supports four message types:

Message Type	Sender	Purpose
HEARTBEAT	Every UAV (every tick)	Broadcasts current state (position, velocity, battery, fault, availability, role) to all peers
HANDOFF_REQUEST	Degraded UAV	Sent when a UAV detects it can no longer complete its mission (low battery, fault). Inc
HANDOFF_BID	Healthy Peer	Response to a HANDOFF_REQUEST. Contains bidding UAV's battery level, distance
HANDOFF_ACCEPT	Original UAV	Confirms task transfer to the winning bidder. Includes remaining waypoints, task conte

Table 3: SwarmBus Message Types

The SwarmBus also manages: **dynamic UAV registration** for horizontal scaling (adding new drones at runtime), **active fault injection** tracking (for testing), **ROS 2 publisher initialization** for live hardware integration, and **MAVLink connection management** for QGroundControl (QGC) compatibility.

3.4 Control Barrier Functions (CBF-QP Safety Filter)

Control Barrier Functions provide **mathematically provable safety guarantees**. The CBF module (`step76_control_barrier_function.py`) implements a QP-based safety filter that takes the agent's proposed velocity command and projects it onto the nearest safe action.

The CBF enforces four barrier constraints simultaneously:

Barrier	Mathematical Formulation	Purpose
Ground Impact	$h_{alt}(z, vz) = (z - z_{min}) - (vz^2 / 2a_{max}) \geq 0$	Ensures the UAV always has enough braking distance before hitti
Speed Limit	$h_{speed}(v) = v_{max}^2 - (vx^2 + vy^2 + vz^2) \geq 0$	Prevents the UAV from exceeding maximum kinetic speed.
Swarm Separation	$h_{swarm}(p_i, p_j) = \|\|p_i - p_j\|\|^2 - d_{min}^2 \geq 0$	Maintains minimum separation distance (2.5m) between all UAV p
NFZ Geofence	$h_{nfz}(p, c) = \|\|p - c\|\|^2 - r_{nfz}^2 \geq 0$	Repels UAV from No-Fly Zone centers. Uses repulsive barrier der

Table 4: CBF Barrier Constraints

The CBF-QP solver minimizes $\|\mathbf{u} - \mathbf{u}_{nom}\|^2$ (deviation from desired velocity) subject to all barrier constraints, using SciPy's quadratic programming solver. This means the UAV follows its intended path as closely as possible while being mathematically guaranteed to never violate safety constraints.

3.5 Machine Learning FDI Model

The Fault Detection and Isolation (FDI) component uses a pre-trained **ExtraTrees (Extremely Randomized Trees) Classifier** to detect sensor faults from telemetry features in real-time.

Property	Value
Algorithm	ExtraTrees Classifier (Ensemble of Randomized Decision Trees)
Training Data	PX4 SITL flight recordings — normal and fault-injected runs
Feature Set	Statistical features from windowed sensor streams (mean, std, range, etc.)
Classes (6)	Normal, GPS Failure, Compass Failure, Accelerometer Failure, Barometer Failure, Gyroscope Failure
Model File	step70_clean_fdi_model.joblib
Feature Names	step70_feature_names.joblib
Inference	Real-time: features extracted every tick, classified in <1ms

Table 5: FDI ML Model Specifications

4. No-Fly Zone Geofencing & Dynamic Rerouting

4.1 NFZ Configuration

The system defines two No-Fly Zones representing restricted airspace over sensitive areas. These zones are circular with defined center coordinates and radii:

Property	NFZ-Alpha (Hospital)	NFZ-Bravo (Airport)
Center (x, y)	(50.0, 60.0) meters	(–15.0, 90.0) meters
Core Radius	18 meters	22 meters
Buffer Radius (Guardrail)	28 meters (core + 10m)	32 meters (core + 10m)
Hard Wall Clamp Radius	23 meters (core + 5m)	27 meters (core + 5m)
Altitude Ceiling	25 meters	30 meters
Visual Color	Red (2D) / Red Cylinder (3D)	Red (2D) / Red Cylinder (3D)
Buffer Visual	Dashed orange circle (2D) Wireframe cylinder (3D)	Dashed orange circle (2D) Wireframe cylinder (3D)

Table 6: No-Fly Zone Configuration

4.2 Multi-Layer Avoidance Strategy

The system uses a **three-tier defense-in-depth strategy** to ensure UAVs never enter No-Fly Zones. These layers work together, with each layer catching what the previous layer might miss:

Layer	Trigger Distance	Action
Predictive Detection	Buffer radius (28m / 32m from center)	When a UAV's projected flight path intersects the buffer zone, the system calculates a t
Hard Wall Clamp	Core + 5m (23m / 27m from center)	If a UAV somehow gets within 5m of the NFZ core boundary, its position is forcibly clam
CBF-QP Barrier	Core radius (18m / 22m from center)	The Layer 2 CBF-QP solver includes NFZ geofence barriers that generate repulsive vel

Table 7: Three-Tier NFZ Avoidance Strategy

4.3 Dynamic Bypass Waypoint Algorithm

When the predictive detection layer triggers, the bypass algorithm works as follows:

- **Step 1 — Proximity Check:** Calculate distance from UAV to each NFZ center. If distance < buffer radius (28m for Alpha, 32m for Bravo), activate reroute mode.

- **Step 2 — Tangent Vector:** Compute the perpendicular tangent vector to the line connecting the UAV to the NFZ center. This gives the direction to fly 'around' the NFZ.
- **Step 3 — Bypass Waypoint:** Place a temporary bypass waypoint at NFZ center + (normalized_offset × (radius + 14m)) in the tangent direction.
- **Step 4 — Latch & Track:** Once a bypass target is set, the UAV locks onto it (latches) and doesn't recalculate every tick (prevents oscillation). Only releases when within 3m of the bypass point.
- **Step 5 — Resume:** After clearing the bypass waypoint, the UAV resumes navigation toward its original assigned waypoint.

The reroute is visually shown on the dashboard as a **cyan dashed line** from the UAV to its bypass target, with a **pulsing cyan halo** around the rerouting drone and a **diamond marker** at the bypass waypoint location.

5. Two-Layer Guardrail Safety System

The guardrail system acts as a **two-step verification mechanism** — every action proposed by the Agentic Reasoning engine must pass through both layers before execution. This ensures that even if the AI makes an incorrect decision, the safety layers will catch and correct it.

5.1 Layer 1 — Soft Policy Guardrail

Layer 1 is a rule-based policy engine that checks proposed actions against operational constraints:

- **Altitude Bounds:** UAV altitude must remain within 2m – 120m AGL (Above Ground Level).
- **Speed Limits:** Horizontal speed must not exceed 5 m/s.
- **Battery Minimums:** If battery < 15%, force RETURN_TO_HOME regardless of current task.
- **NFZ Proximity:** If UAV is within NFZ buffer zone, action must include rerouting.
- **Action Validation:** Proposed action must be in the allowed set for current fault/energy state.

Layer 1 can **modify or override** the proposed action. For example, if the reasoning engine says 'NOMINAL_CONTINUE' but battery is at 12%, Layer 1 will override to 'RETURN_TO_HOME'. The output status is either **PASSED** (action approved) or **MODIFIED** (action was changed).

5.2 Layer 2 — CBF-QP Hard Safety Filter

Layer 2 is the mathematically rigorous safety filter. It takes the Layer 1-approved velocity command and solves a Quadratic Programming (QP) optimization problem to find the closest safe velocity:

```
minimize    ||u – u_nom||²
subject to  A_cbf · u ≤ b_cbf

Where u_nom is the desired velocity, and A_cbf/b_cbf encode
all barrier constraints (ground, speed, swarm, NFZ).
```

If the proposed velocity is already safe, the CBF returns it unchanged (status: **PASSED**). If it would violate any barrier, the CBF computes the **minimum modification** needed to make it safe (status: **INTERVENED**). The UAV's trajectory is always the closest feasible path to its desired course.

6. Swarm Coordination & Intelligence

6.1 Leader Election Protocol

The swarm uses a **dynamic leader election algorithm** based on a composite fitness score. Every tick, each UAV's fitness is evaluated:

```
fitness_score = battery_pct × 0.4
             + (1.0 − risk_score) × 100 × 0.3
             + link_quality_pct × 0.2
             + (1.0 if no_fault else 0.0) × 10 × 0.1
```

The UAV with the highest fitness score becomes the **Swarm Leader** (marked with ■). Leadership transfers seamlessly — if the current leader develops a fault or low battery, a healthier peer automatically takes over. The leader coordinates task assignments and serves as the reference node for swarm formation.

6.2 Task Delegation & Handoff Protocol

When a UAV becomes degraded (fault detected, low battery, high risk), the **Handoff Protocol** enables graceful task transfer to a healthy peer:

- **Step 1 — HandoffRequest:** Degraded UAV broadcasts its current task, remaining waypoints, and reason for handoff to all peers via SwarmBus.
- **Step 2 — HandoffBid:** Each healthy, available peer evaluates its own capacity (battery, distance to task, fault status) and submits a bid score.
- **Step 3 — HandoffAccept:** The UAV with the best bid (closest + most battery) wins the task. The original UAV transfers task context and authorization.
- **Step 4 — Beacon Mode:** The degraded UAV enters 'Beacon Active (Awaiting Retrieval)' state — it hovers in place, broadcasting its location for physical retrieval while the winning peer continues the mission.

6.3 Inter-Agent Collision Avoidance

Inter-agent collision avoidance is handled at two levels: the SwarmBus shares position data between all agents every tick, and the CBF-QP Layer 2 safety filter includes **swarm separation barriers** that maintain a minimum distance of **2.5 meters** between any two UAVs. If two UAVs are on a collision course, the CBF automatically adjusts their velocities to diverge.

7. Energy & Battery Management

The Energy Monitor node continuously tracks each UAV's battery state and classifies it into four alert levels:

Alert Level	Battery Range	Color	System Response
NOMINAL	> 30%	Green	Normal operations continue. No intervention.
WARNING	15% – 30%	Yellow	Reduced speed recommended. Swarm leader notified. Mission continues with
CRITICAL	5% – 15%	Orange/Red	Forced RETURN_TO_HOME. Mission is aborted. Task handoff initiated to hea
EMERGENCY	< 5%	Red (Flashing)	Immediate EMERGENCY_LAND at current position. UAV enters beacon mode

Table 8: Energy Alert Levels

Battery drain is simulated realistically: base drain of ~0.02% per tick plus speed-dependent drain. Fault conditions accelerate drain. The system also detects **anomalous battery drain** patterns that could indicate a faulty battery or power system.

8. Risk Assessment & Agentic Reasoning

The Risk Assessment node computes a **composite risk score** (0.0 to 1.0) by combining multiple risk factors with weighted contributions:

- **Fault Detection Risk (40%):** Non-normal FDI classification adds 0.4 to risk score.
- **Battery Risk (25%):** Scales inversely with battery percentage. Below 20% adds significant risk.
- **Speed Anomaly (15%):** Excessive speed (> 4 m/s) or very low speed (< 0.5 m/s) adds risk.
- **Altitude Risk (10%):** Very low altitude (< 3m) or very high altitude (> 100m) adds risk.
- **NFZ Proximity (10%):** Being near a No-Fly Zone buffer boundary adds risk.

The Agentic Reasoning engine maps the composite risk score to **flight actions**:

Risk Tier	Risk Score	Typical Action
LOW	0.0 – 0.3	NOMINAL_CONTINUE — Continue mission normally.
MODERATE	0.3 – 0.5	REDUCE_SPEED — Slow down, increase monitoring frequency.
HIGH	0.5 – 0.7	ALTITUDE_HOLD — Hold position, attempt recovery.
CRITICAL	0.7 – 1.0	RETURN_TO_HOME or EMERGENCY_LAND — Abort mission.

Table 9: Risk-to-Action Mapping

9. Mission Control Web Dashboard

The Mission Control Dashboard is a **single-page web application** served by a Tornado web server (`mission_control.py`). It provides real-time visualization and control of the entire swarm mission.

9.1 3D Three.js Visualization

- Full 3D scene rendered with Three.js and OrbitControls for free rotation/zoom.
- Each UAV is shown as a colored sphere with a glowing halo indicating its status.
- Flight paths are drawn as colored lines connecting waypoints.
- No-Fly Zones are rendered as semi-transparent red cylinders with wireframe buffer zones.
- Ground grid provides spatial reference for the 200m × 200m operational area.
- UAV labels with ID and altitude float above each drone.

9.2 2D Radar Map with Zoom/Pan

- Top-down 2D Canvas-based radar view for precise position monitoring.
- **Mouse wheel zoom** — zooms in/out centered on cursor position. Everything scales proportionally (UAV markers, NFZ circles, buffer zones, waypoints, labels, routes).
- **Click-and-drag pan** — moves the viewport to explore different areas of the operational zone.
- NFZ-Alpha and NFZ-Bravo shown as filled red circles with labeled names.
- Outer buffer zones shown as dashed orange rings with appropriate radii.
- Planned routes drawn as colored dashed lines connecting waypoints.
- Dynamic reroute vectors shown as cyan dashed lines when a UAV is actively bypassing an NFZ.
- UAV callsigns and altitude labels rendered next to each drone marker.

9.3 Telemetry Cards & Live Status

Each UAV has a dedicated telemetry card showing:

- **Position:** Current (x, y, z) coordinates in meters.
- **Battery:** Percentage with color-coded indicator (green/yellow/red).
- **FDI Status:** Current fault classification from the ML model.
- **Risk Score:** Composite risk value with tier label (LOW/MODERATE/HIGH/CRITICAL).
- **Flight Action:** Current action being executed (NOMINAL_CONTINUE, REROUTING, RTL, etc.).
- **Guardrail Status:** L1 and L2 guardrail results (PASSED/MODIFIED/INTERVENED).
- **Swarm Role:** Current role (Swarm Leader ■, Relay/Escort, Survey, Reserve).
- **Task Progress:** Waypoints completed / total waypoints.
- **Reroute Badge:** Shows '■ REROUTING' when UAV is actively bypassing an NFZ.

9.4 Fault Injection Console

The dashboard includes a command console for injecting faults in real-time during a mission. This allows testing the system's response to various failure scenarios:

- **Inject Fault:** `fault UAV-2 GPS` — Injects a GPS failure on UAV-2.
- **Clear Fault:** `clear UAV-2` — Clears injected fault on UAV-2.
- **Clear All:** `clear all` — Clears all injected faults.
- **Set Battery:** `batt UAV-2 14` — Sets UAV-2 battery to 14%.
- Available fault types: GPS, Compass, Accelerometer, Barometer, Gyroscope.

10. Terminology & Glossary

10.1 Flight Actions

Action	Description
NOMINAL_CONTINUE	Normal flight — UAV is following its planned route to the next waypoint. All systems healthy, no inter
REDUCE_SPEED	Speed reduced due to moderate risk (sensor anomaly or approaching NFZ buffer). Mission continue
ALTITUDE_HOLD	UAV holds current altitude and position while the system diagnoses a fault or waits for conditions to
RETURN_TO_HOME (RTL)	Mission aborted. UAV navigates back to its launch/home position due to critical battery, persistent fa
EMERGENCY_LAND	Immediate landing at current position. Used when battery is critically low (< 5%) or a catastrophic fau
DELEGATE_TO_SWARM	Task is being transferred to a peer UAV via the Handoff Protocol. This UAV will enter beacon mode
REROUTING	UAV is actively navigating around a No-Fly Zone using a computed bypass waypoint. Shown as cya
Beacon Active (Awaiting Retrieval)	UAV has completed its handoff, has very low battery, or has landed. It hovers/sits in place broadcast

Table 10: Flight Action Definitions

10.2 FDI Fault Labels

Label	Description
Normal (No Failure)	All sensors operating within expected parameters. No fault detected.
GPS Failure	GPS receiver malfunction — position/velocity data unreliable. Could be jamming, signal loss, or harc
Compass Failure	Magnetometer fault — heading data corrupted. Can cause navigation drift. Often caused by electrom
Accelerometer Failure	IMU accelerometer fault — acceleration measurements unreliable. Affects attitude estimation and ve
Barometer Failure	Pressure sensor fault — altitude readings inaccurate. Critical for altitude hold and terrain following.
Gyroscope Failure	IMU gyroscope fault — angular velocity measurements corrupted. Affects attitude stabilization and r

Table 11: FDI Fault Classification Labels

10.3 Energy Alert Levels

See Table 8 in Chapter 7 for the complete energy alert level definitions (NOMINAL, WARNING, CRITICAL, EMERGENCY).

10.4 Guardrail Status Codes

Status	Layer	Meaning
PASSED	L1 or L2	The proposed action/velocity passed safety checks without modification.

Status	Layer	Meaning
MODIFIED	L1	The soft policy guardrail changed the proposed action to meet operational rules (e.g., forced R
INTERVENED	L2 (CBF)	The CBF-QP safety filter modified the velocity vector to prevent a barrier violation (collision, NF

Table 12: Guardrail Status Codes

10.5 Swarm Roles

Role	Description
■ Swarm Leader	UAV with highest fitness score. Coordinates task assignments, serves as reference node for formation,
Relay / Escort	Communication relay node. Extends mesh network range and escorts other UAVs through contested or
Survey Bravo	Dedicated survey role for Sector Bravo. Performs reconnaissance and data collection along its assigned
Swarm Reserve	Standby UAV available for emergency task pickup. Performs perimeter patrol while ready to accept han

Table 13: Swarm Role Definitions

11. Technology Stack

Technology	Purpose	Details
Python 3.x	Core Language	All backend simulation, ML inference, and agent logic.
LangGraph	Agentic AI Framework	State graph pipeline for multi-node agent processing. Each UAV runs an independent
NumPy	Numerical Computing	Vector math for position updates, distance calculations, barrier function evaluation.
SciPy	Optimization	QP solver for Control Barrier Function safety filtering.
scikit-learn	Machine Learning	ExtraTrees classifier for FDI. Model trained offline, loaded via joblib for real-time infere
joblib	Model Serialization	Loads pre-trained FDI model and feature names from .joblib files.
Tornado	Web Server	Asynchronous HTTP/WebSocket server for Mission Control dashboard. Handles RES
Three.js	3D Visualization	WebGL-based 3D scene rendering in the browser. Shows UAVs, flight paths, NFZ cyl
HTML5 Canvas	2D Radar	Canvas-based 2D top-down radar with zoom/pan controls.
JavaScript (ES6)	Frontend	Dashboard UI logic, WebSocket client, telemetry rendering.
ROS 2 (Optional)	Robot Framework	Publishes UAV state to ROS 2 topics for integration with real PX4 autopilot and senso
MAVLink (Optional)	Protocol	Standard UAV communication protocol. Enables integration with QGroundControl (QG
asyncio	Async Runtime	Python async/await for concurrent swarm tick execution.
CSS3	Styling	Dashboard styling with dark theme, responsive layout, and animations.

Table 14: Technology Stack

12. File Structure & Codebase

File / Directory	Size	Description
step80_multiagent_swarm.py	~2500 lines	Core simulation engine. Contains SwarmBus, SwarmAgentState, all 10 pipel
step76_control_barrier_function.py	~220 lines	CBF-QP safety filter module. Implements ground, speed, swarm separation,
step70_build_and_train_clean.py	—	Training script for the ExtraTrees FDI classifier. Processes PX4 flight recordi
step70_clean_fdi_model.joblib	~2 MB	Serialized pre-trained ExtraTrees classifier model.
step70_feature_names.joblib	< 1 KB	List of feature names expected by the FDI model.
mission_control.py	~300 lines	Tornado web server. Serves dashboard HTML, REST API endpoints (/api/tel
web_gui/index.html	~1750 lines	Single-page dashboard: 3D Three.js scene, 2D Canvas radar, telemetry card
run_mission_control.sh	~43 lines	One-click launcher script. Kills stale processes, starts mission control server,
master_system_report.json	Dynamic	Live telemetry export (updated every tick). Contains all agent states, swarm s
step71_live_px4_fdi.py	—	Live PX4 SITL FDI integration script for real-time fault detection.
step72_generate_evaluation_report.py		Generates evaluation plots and confusion matrices for the FDI model.
step73_autonomous_safety_governor.py		Standalone autonomous safety governor prototype.
step75_agentic_safety_governor.py	—	Agentic AI safety governor with LangGraph integration.
step_auto_*.csv / step*.csv	Multiple	PX4 flight recording datasets for normal and fault-injected flights. Used for FD
fdi_env/	Directory	Python virtual environment with all project dependencies installed.

Table 15: Project File Structure

13. How to Run the System

13.1 Prerequisites

- **Python 3.8+** with pip.
- Virtual environment at `fdi_env/` with dependencies: langgraph, numpy, scipy, scikit-learn, joblib, tornado.
- A modern web browser (Chrome, Firefox, Edge) for the dashboard.
- Optional: ROS 2 Humble/Iron for live hardware integration.

13.2 Quick Start (One Command)

Navigate to the project directory and run:

```
cd /home/samiksha/uav_fdi_data
./run_mission_control.sh
```

This will: (1) kill any stale processes, (2) start the Tornado web server on port 8080 (falls back to 8081 if 8080 is occupied), and (3) open the Mission Control Dashboard in your browser.

13.3 Manual Start

If you prefer to start components individually:

```
# Step 1: Activate the virtual environment
source fdi_env/bin/activate

# Step 2: Start Mission Control web server
python mission_control.py &

# Step 3: Open browser to http://localhost:8080

# Step 4: Click 'Launch Mission' in the dashboard
```

13.4 Terminal-Only Mode (No Dashboard)

You can also run the swarm simulation directly in the terminal with the ANSI dashboard:

```
source fdi_env/bin/activate
python step80_multiagent_swarm.py
```

This launches the rich terminal dashboard showing side-by-side agent status cards, swarm overview, and inter-agent communication log with color-coded outputs.

13.5 Fault Injection During Flight

While the mission is running, use the **Fault Injection Console** in the dashboard or the terminal to test system responses:

```
# In terminal (stdin commands):
fault UAV-2 GPS      # Inject GPS failure on UAV-2
fault UAV-3 Compass  # Inject Compass failure on UAV-3
batt UAV-1 12        # Set UAV-1 battery to 12%
clear UAV-2          # Clear fault on UAV-2
```

```
clear all            # Clear all injected faults
```

14. Future Enhancements

- **Real Hardware Integration:** Deploy on actual PX4-powered drones via ROS 2 Micro-XRCE-DDS bridge. The agent pipeline is designed to swap in real sensor data seamlessly.
- **Dynamic NFZ Updates:** Support for receiving real-time NOTAM (Notice to Airmen) updates and dynamically adding/removing No-Fly Zones during flight.
- **Multi-Fault Detection:** Extend the FDI model to detect simultaneous faults on multiple sensors (currently detects single-fault scenarios).
- **Path Planning Optimization:** Integrate A* or RRT* path planning for optimal NFZ bypass routes instead of tangent-based bypass waypoints.
- **Weather Integration:** Factor in real-time wind and weather data for more realistic battery drain simulation and flight planning.
- **Swarm Formation Control:** Implement formation flying (V-formation, line abreast) with distributed consensus algorithms.
- **Secure Communication:** Add encryption and authentication to SwarmBus messages for cybersecurity in real-world deployments.
- **3D NFZ Volumes:** Extend NFZ from 2D circles to full 3D volumes with altitude ceilings and irregular shapes (polygon geofences).
- **Mission Replay:** Add mission replay capability using recorded telemetry CSV data for post-mission analysis and training.
- **LLM-Enhanced Reasoning:** Integrate large language model reasoning in the Agentic Reasoning node for natural language mission reports and adaptive decision-making.

15. Conclusion

The UAV Swarm Fault Detection & Isolation system demonstrates a comprehensive, production-ready architecture for autonomous multi-agent drone operations. By combining **LangGraph-based Agentic AI** for intelligent decision-making, **Control Barrier Functions** for mathematically guaranteed safety, and **Machine Learning** for real-time fault detection, the system achieves a level of autonomy and safety assurance that meets the demands of real-world UAV swarm deployments.

Key achievements of this project include:

- A **10-node LangGraph pipeline** executing independently on each UAV agent every tick.
- **6-class sensor fault detection** using ExtraTrees ML with sub-millisecond inference.
- **Three-tier No-Fly Zone avoidance** (predictive rerouting, hard wall clamp, CBF barrier).
- **Two-layer guardrail safety system** providing both policy enforcement and mathematical safety guarantees.
- **Dynamic swarm coordination** with leader election, task handoff, and peer collision avoidance.

- **Real-time Mission Control dashboard** with 3D visualization, 2D radar, and live telemetry.
- **Horizontal scaling** — dynamically add or remove UAVs from the swarm at runtime.
- **ROS 2 and MAVLink integration readiness** for deployment on real PX4-powered drones.

The system is designed to be extensible, with clear separation between the intelligence layer (LangGraph agents), safety layer (CBF guardrails), communication layer (SwarmBus), and visualization layer (web dashboard). This modular architecture makes it straightforward to add new capabilities, integrate with real hardware, or adapt the system for different autonomous vehicle domains beyond UAVs.

— End of Documentation —
Generated on October 23, 2026 at 01:20 PM